Computer Vision:

Homework-10

Bilal Ahmed
ahmedb@purdue.edu

1 Theoretical Questions

Summarize in your own words your overall understanding of the notion "overfitting to the training data"

Machine learning algorithms are trained on the training data and their performance is evaluated on held out data that is independent of the training data but comes from the same distribution as the training data. The idea is for machine learning algorithm to learn to detect examples based on the features using a strategy that is general enough to work on the held out data or any new example not seen before by the model. The model is said to overfit the training data when it memories the training examples and is able to detect them but fails to perform on the novel examples. An over-parameterized model having many parameters has the capacity to memorize the training examples and overfit to the training data. This is undesirable so regularization is often used to impose some constraint or prior knowledge on the learned solution to avoid overfitting.

"Reparameterization Trick" in VAE

Variational autoencoders learns means and standard deviation of latent distribution. Then decoder takes in a sample from that distribution and reconstructs the original input. This sampling from latent distribution poses a challenge for gradient based learning. During gradient descent loss is computed at the output of the model and is back propagated to all the parameters (weights) of the VAE. This works fine for parameters of decoder but for parameters of encoder, this gradient back propagation has to pass through the sampler of latent distribution. This is not possible and hence poses a challenge for learning the weights of encoder. Reparameterization trick was used in VAE paper, that re-parameterizes latent distribution as a random variable having standard normal distribution scaled by leaned standard deviation and shifted by learned mean. This way the randomness of sampling step is taken out of the way for gradient back propagation and learning the parameters of encoder is made possible.

2 Face Recognition using PCA and LDA

Goal of this task is to implement face recognition on FacePix dataset using dimensionality reduction by principle component analysis (PCA) and linear discriminant analysis (LDA). As a first step, a low dimensional representation of data is obtained and then detection is done using nearest neighbor (NN) algorithm. First I'll explain the dimensionality reduction using PCA and LDA.

Principle Component Analysis (PCA)

PCA is a linear dimensionality reduction algorithm where a high dimensional data is represented by small number of dimensions along the highest variance directions. Here is the step by step procedure;

- Read each image, flatten to get a vector and normalize to get unit norm.

- Stack all images in the training set to get a data matrix

$$X = \Big[\mathbf{x}_1 - \mathbf{m} \;\Big|\; \mathbf{x}_2 - \mathbf{m} \;\Big|\; \cdots \;\Big|\; \mathbf{x}_N - \mathbf{m}\Big]$$

where

$$\vec{\mathbf{m}} = \frac{1}{N}\sum_{i=1}^{N}\vec{\mathbf{x}_i}$$

is the mean of all image vectors and $\mathbf{X} \in \mathbb{R}^{d\times N}$, d is the size of each image vector and N is the total number of images.

- We can compute the covariance matrix as;

$$\mathbf{C} = \mathbf{X}\mathbf{X}^T$$

here $\mathbf{C} \in \mathbb{R}^{d\times d}$. In case there are large number of dimensions d of each image, as is mostly the case, computing eigen vectors of C along maximum variance directions is very costly.

- We can overcome this problem using computational trick. Instead of finding eigen vectors of $\mathbf{X}\mathbf{X}^T$ that has large dimensions, we can compute eigen vectors of $\mathbf{X}^T\mathbf{X}$ that is $N \times N$. Then we can get the eigen vectors of C as follows;

$$\mathbf{X}^T\mathbf{X}w = \lambda w$$

Here w is the eigen vector of $\mathbf{X}^T\mathbf{X}$.

$$\mathbf{X}\mathbf{X}^T\mathbf{X}w = \lambda\mathbf{X}w$$

This implies the eigen vector of $\mathbf{X}\mathbf{X}^T$ is $\mathbf{X}w$

- We select the k eigen vectors having the largest eigen values, to get the matrix $\mathbf{W_k} \in \mathbb{R}^{d\times k}$ that defines the PCA subspace.

- We can project each image vector to this subspace as follows;

$$\vec{\mathbf{y}} = \hat{W_k}^T(\vec{\mathbf{x}} - \vec{\mathbf{m}})$$

Linear Discriminant Analysis (LDA)

LDA projects high dimensional vectors in a subspace such that between class variance is maximized and within class variance is minimized.

- Calculate class specific mean $\vec{m_i}$ and global mean $\vec{m}$ using

$$\vec{\mathbf{m}_i} = \frac{1}{|C_i|}\sum_{k=1}^{|C_i|}\vec{\mathbf{x}^{\mathbf{i}}}_k$$

$$\vec{\mathbf{m}} = \frac{1}{|C|}\sum_{i=1}^{|C|}\vec{\mathbf{m}_\mathbf{i}}$$

- Between class variance is defined as variance of class specific means;

$$\mathbf{S_B} = \frac{1}{|C|}\sum_{i=1}^{|C|}(\vec{m_i} - \vec{m})(\vec{m_i} - \vec{m})^T$$

- Within class variance is given as average variance of all classes;

$$\mathbf{S_W} = \frac{1}{|C|}\sum_{i=1}^{|C|}\frac{1}{|C_i|}\sum_{k=1}^{|C_i|}(\vec{x}_k^i - \vec{m}_i)(\vec{x}_k^i - \vec{m}_i)^T$$

- Eigen directions that maximum between class variance and minimize within class variance can be obtained by minimizing the objective functions given below and call Fisher Discriminant Function;

$$J(\vec{w}) = \frac{\vec{w}^T\mathbf{S_B}\vec{w}}{\vec{w}^T\mathbf{S_W}\vec{w}}$$

- Solution can be found by solving generalized eigen-value problem

$$\mathbf{S_B}\vec{w} = \lambda\mathbf{S_W}\vec{w}$$

  which can be done by finding the eigen decomposition of $S_W^{-1}S_B$

An efficient implementation of LDA is provided by y Hua Yu and Jie Yang ("A Direct LDA Algorithm for High-Dimensional Data...." in Pattern Recognition, 2001)

Accuracy of PCA vs LDA

Since PCA finds directions of high variance, it can discard class discriminatory directions whereas LDA makes sure to retain maximum class discriminator directions are retained. That is why PCA performs poorly than LDA especially for smaller sub-space dimensions. As we increase the number of dimensions of sub-space accuracy of both PCA and LDA converges to the same level.

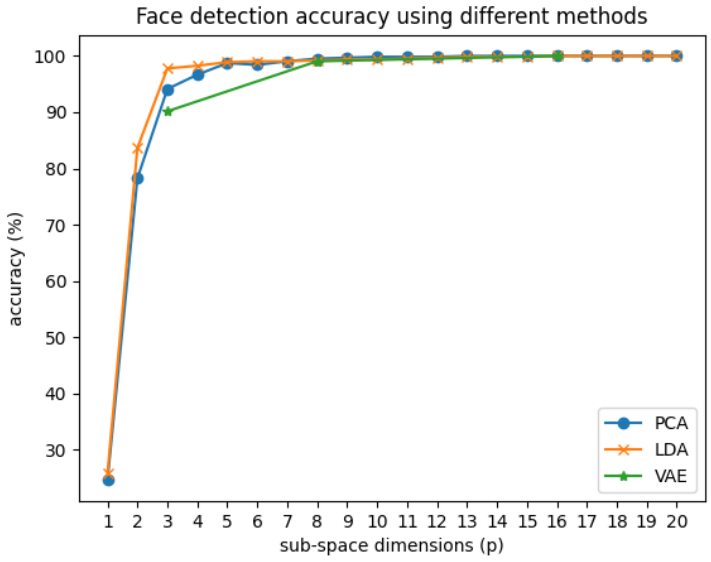


Figure 1: Accuracy of face detection using PCA, LDA, and VAE features as a function of the number of dimensions.

Accuracy of Auto-encoder features

Auto-encoder is a neural network that projects input features to low dimensional space and then reconstructs the original dimensions back. Encoder of auto-encoder can also be used to get low

dimension features for face recognition. These low dimensional features are learned by autoencoder over the course of training. Autoencoder learns those features such that they carry information that allows decoder to recover the image back. But we can see the accuracy of autoencoder is lower than both PCA and LDA for p=3 where as it converges to the maximum performace of PCA and LDA from p=8 and higher. So for lower p autoencoder learned features are not as good as LDA or PCA for classification.

Umap plots for PCA, LDA and Autoencoder

Umap embedding provides a way to visualize the features of images. I am added plots of umap embeddings for p=3,8,16. Looking at these plots, it seems the LDA features are better at segregating class specific features for p=3 in comparison to PCA and autoencoder. Whereas for higher p, there is not much difference in umap embedding of features.

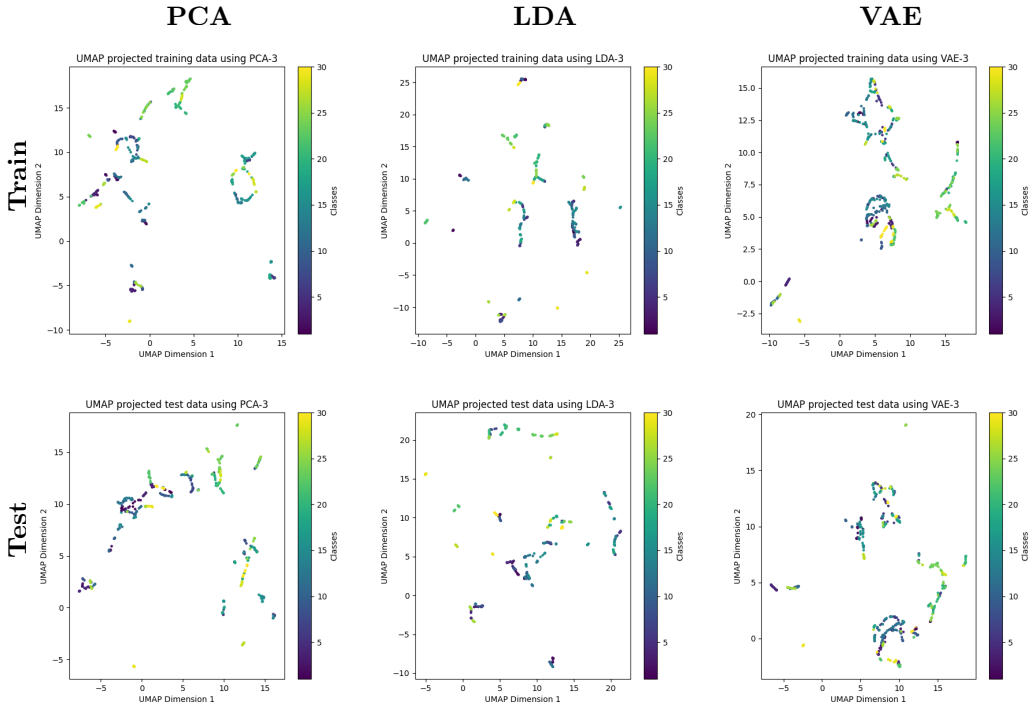


Figure 2: Visualization of embeddings using 3 dimensions. The first row represents training set embeddings, and the second row represents test set embeddings. Features are shown for PCA, LDA, and VAE in respective columns.

3 Object Detection using Cascaded AdaBoost Classifiers

AdaBoost algorithm based classifier selects features that are good at classifying examples miss-classified by already selected features. In this these successive classifiers build on the performance of earlier classifiers. This way we can design a classifier with arbitrary false-positive rate or false negative rate. Here is the outline of the steps I implemented;

- For each image, vertical and horizontal haar features are computed with varying sizes of windows starting from 2 all the way to the full width or height.
- At each iteration, weak classifier is selected as a feature that maximizes the classification accuracy.

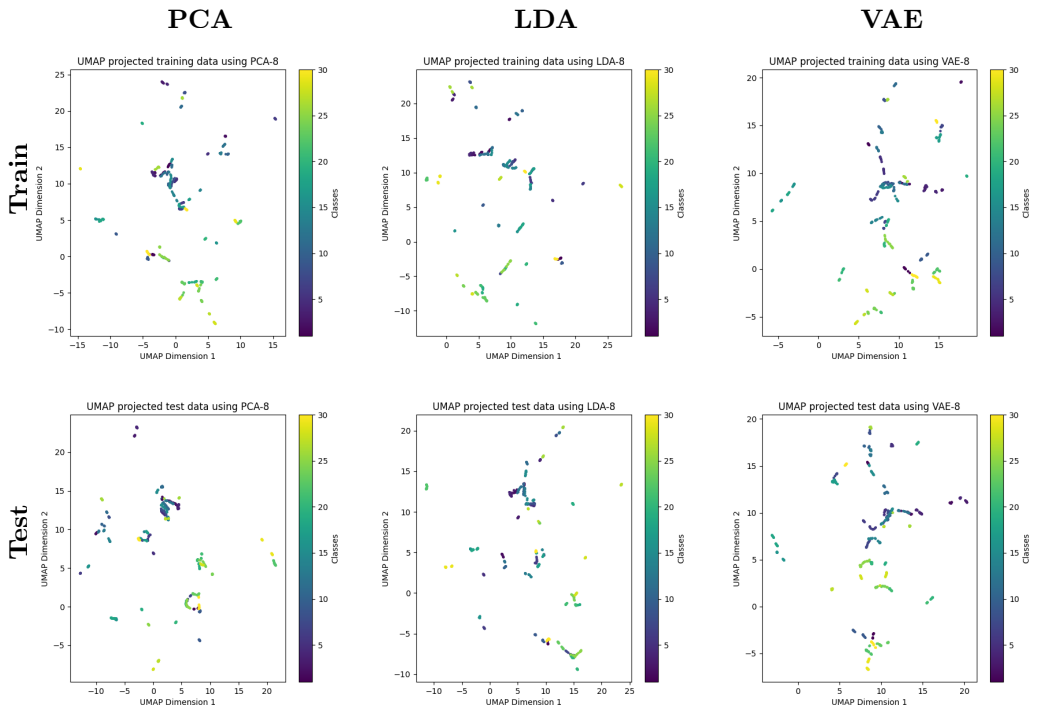


Figure 3: Visualization of embeddings using 8 dimensions. The first row represents training set embeddings, and the second row represents test set embeddings. Features are shown for PCA, LDA, and VAE in respective columns.

- For the selected feature, threshold and polarity, weighted classification error is calculated as;

$$\epsilon_t = \frac{1}{2}\sum_{i=1}^{N} D_t(x_i) \cdot |h_t(x_i) - y_i|$$

  where $D_t(x_i)$ is the probability of sample x_i to be selected at the current level. Note that this probability is initialized to uniform and adjusted later on based on performance of weak classifiers.

- For the selected weak classifier, trust factor is calculated based on its error;

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

- Probability distribution of samples or weighting distribution is updated as follows;

$$D_{t+1}(x_i) = \frac{D_t(x_i)e^{-\alpha_t y_i h_t(x_i)}}{Z_t}$$

  where

$$Z_t = \sum_{i=1}^{N} D_t(x_i)e^{-\alpha_t y_i h_t(x_i)}$$

  Note than these expressions are defined for the class labels of negative class as -1, such that $y_i h_t(x_i) = 1$ for correctly classified labels and $y_i h_t(x_i) = -1$ for miss-classified labels. I used the label of 0 for negative class, so I explicitly set this product equal to -1 for miss-classified labels and equal to 1 for correctly classified classes.

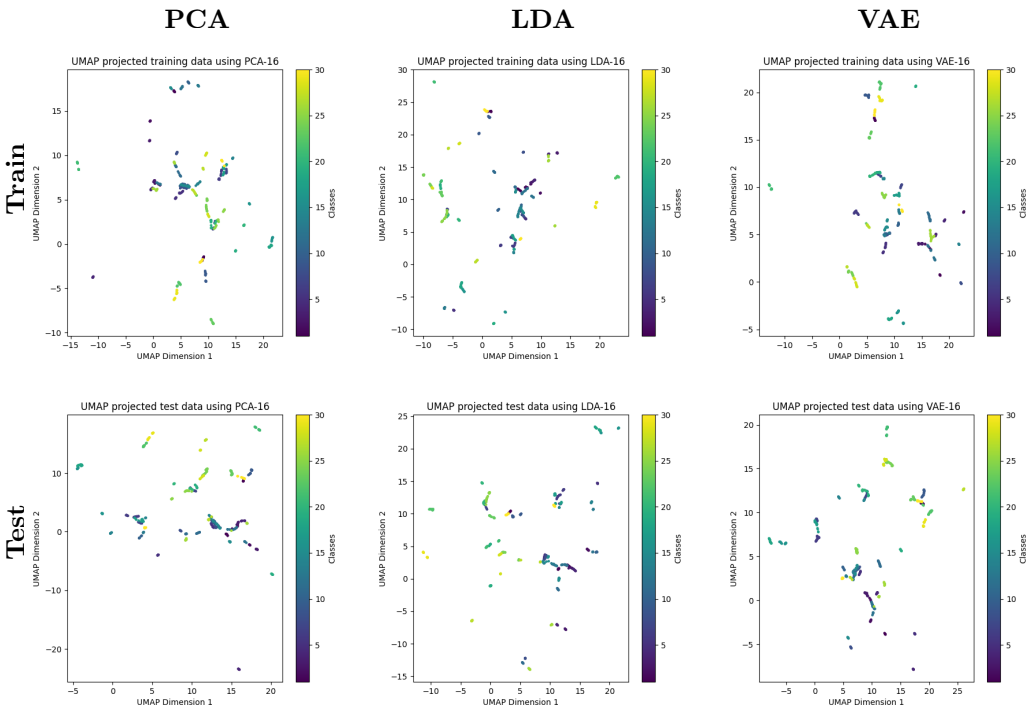


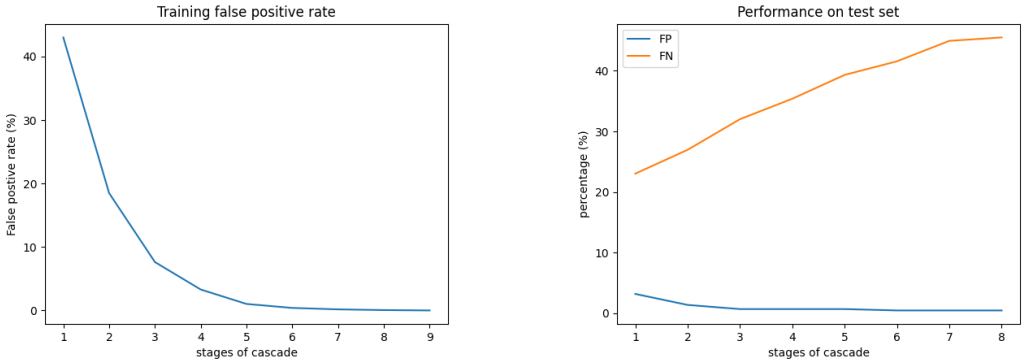
Figure 4: Visualization of embeddings using 16 dimensions. The first row represents training set embeddings, and the second row represents test set embeddings. Features are shown for PCA, LDA, and VAE in respective columns.

- Then using the new distribution of weights, next weak classifier is selected and the process is repeated until desired performance is achieved.
- Once enough number weak classifier are selected such that desired true positive rate is achieved, class predictions are made using;

$$H(x) = \left(\sum_{t=1}^{N} \alpha_t h_t(x) \right)$$

  and predicting H(X) to threshold for predicted class label. At test time I set this threshold equal to 0.5.
- The process explain so far, gives us AdaBoost classifier having the desired true positive rate and false positive rate. Let's say true positive rate is 0.99 and false positive rate is 0.3. We can repeat this process many times and keep adding multiple stages of adaBoost in a cascade to lower the false positive rate. For example cascading 10 stages, would lower the false positive rate to $(0.3)^{10}$ but it will also lower the true positive rate but rather slowly to $(0.99)^{10}$
- At test time, an image is labeled as negative when any one of the stage labels it as negative. But in order for an example to be labeled as positive all the stages of cascade must label it as positive.

5 shows the false positive rate during training and testing. It must be noted that as we add more and more stages, false positive rate goes to very small values but false negative rate also starts to increase to significantly higher values.



(a) Training False Positive Rate (b) Testing False Positive Rate

Figure 5: False positive rate for cascaded AdaBoost during training and testing for car detection.

Python Code

```python
import cv2

import umap.umap_ as umap
import matplotlib.pylab as plt
import numpy as np
import os

import numpy as np
import torch
from torch import nn, optim
from PIL import Image
from torch.autograd import Variable
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

torch.set_grad_enabled(False)


import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(levelname)s - %(message)s',  # Log message format
)

logger = logging.getLogger(__name__)


def read_image_from_path(image_path):
        """Reads images from the subdir"""
        img_bgr = cv2.imread(image_path)
        # Convert the image from BGR to RGB format
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
        return img_rgb


class FaceClassifier:
    def __init__(self, data_dir, output_dir) -> None:
```

```
        self.data_dir = data_dir
        self.output_dir = output_dir
        # reading the training images
        self.train_images, self.train_y = self.read_images(True)
        self.train_x = self.normalize(
            self.train_images.reshape(self.train_images.shape[0], -1).T
            )

        # reading the test images
        self.test_images, self.test_y = self.read_images(False)
        self.test_x = self.normalize(
            self.test_images.reshape(self.test_images.shape[0], -1).T
            )

    def get_PCA_space(self, k):
        """Return the vectors along the PCA subspace"""
        X = self.train_x

        s, Q = self.eigen_decomposition(X)
        W_k = Q[:,:k]
        return W_k

    def get_LDA_space(self, k):
        X = self.train_x
        y = self.train_y
        C = np.unique(y)
        class_means = []
        for c_i in C:
            class_examples = X[:,np.where(y == c_i)].squeeze()  # (
                num_features, Ni)
            class_means.append(np.mean(class_examples, axis=-1))

        s, Q = self.eigen_decomposition(np.array(class_means).T)
        # discarding the smallest eigen value and corresponding eigen vector
            ...
        Z = Q[:,:-1]@np.diag(np.sqrt(s[:-1]))

        # computational trick for the second eigen decomposition...
        x_mean = np.mean(X, axis=1)
        X = X - x_mean[:, None]
        G = Z.T@X
        s, U = self.eigen_decomposition(G)
        U_hat = U[:,:k]
        W_k = Z@U_hat
        return W_k

    def NN_predictor(self, W_k):
        """Uses nearest neighbor algorithm to predict class label for the
        test_x, projects data to the sub_space before computing distances.
        """
        X = self.train_x
        X_test = self.test_x

        x_mean = np.mean(X, axis=1)
        X = X - x_mean[:, None]

        # test_mean = np.mean(X_test, axis=0)
        X_test = X_test - x_mean[:, None]

```

```
        train_w = W_k.T @X
        test_w = W_k.T @X_test

        return self.NN(train_w, self.train_y, test_w)
        # # predicts class label using the nearest neighbor based on Euclidean distance...
        # diff = test_w[:,:,None] - train_w[:,None,:]
        # predicted_y = self.train_y[np.argmin(np.linalg.norm(diff, axis=0), axis=-1)]
        # return predicted_y

    @staticmethod
    def NN(train_x, train_y, test_x):
        """Nearest neighbor algorithm

        Args:
            train_x: shape=(features, examples)
            train_y: shape=(examples,)
            test_x: shape=(features, examples)

        """
        # predicts class label using the nearest neighbor based on Euclidean distance...
        diff = test_x[:,:,None] - train_x[:,None,:]
        predicted_y = train_y[np.argmin(np.linalg.norm(diff, axis=0), axis=-1)]
        return predicted_y

    def compute_accuracy(self, predicted_y):
        return 100*np.sum(predicted_y== self.test_y)/self.test_y.size


    def read_images(self, train=True):
        if train:
            split = 'train'
        else:
            split = 'test'

        filenames = os.listdir(os.path.join(self.data_dir, split))
        images = []
        labels = []
        for filename in filenames:
            img = read_image_from_path(os.path.join(self.data_dir, split, filename))
            images.append(img)
            labels.append(int(filename.split('_')[0]))
        return np.array(images).astype(np.float32), np.array(labels).astype(np.float32)

    def visualize_embeddings(self, k=2):

        W_k = self.get_PCA_space(k=k)
        predicted_y = self.NN_predictor(W_k)
        pca_train_data = W_k.T @ self.train_x
        pca_test_data = W_k.T @ self.test_x

        self.plot_umap_embeddings(pca_train_data.T, self.train_y)
        plt.title(f'UMAP projected training data using PCA-{k}')
```

```
        plt.savefig(os.path.join(self.output_dir, f'pca_umap_k_{k}_train.png'))

        self.plot_umap_embeddings(pca_test_data.T, predicted_y)
        plt.title(f'UMAP projected test data using PCA-{k}')
        plt.savefig(os.path.join(self.output_dir, f'pca_umap_k_{k}_test.png'))

        W_k = self.get_LDA_space(k=k)
        predicted_y = self.NN_predictor(W_k)
        lda_train_data = W_k.T @ self.train_x
        lda_test_data = W_k.T @ self.test_x

        self.plot_umap_embeddings(lda_train_data.T, self.train_y)
        plt.title(f'UMAP projected training data using LDA-{k}')
        plt.savefig(os.path.join(self.output_dir, f'lda_umap_k_{k}_train.png'))

        self.plot_umap_embeddings(lda_test_data.T, predicted_y)
        plt.title(f'UMAP projected test data using LDA-{k}')
        plt.savefig(os.path.join(self.output_dir, f'lda_umap_k_{k}_test.png'))



    @staticmethod
    def plot_umap_embeddings(data, y, cmap='viridis', s=5, ax=None):
        umap_model = umap.UMAP()
        X_embedded = umap_model.fit_transform(data)

        if ax is None:
            fig, ax = plt.subplots(figsize=(6, 6))
        scatter = ax.scatter(X_embedded[:, 0], X_embedded[:, 1], c=y, cmap=cmap, s=s)
        plt.colorbar(scatter, label='Classes')
        ax.set_xlabel('UMAP Dimension 1')
        ax.set_ylabel('UMAP Dimension 2')
        return ax


    @staticmethod
    def eigen_decomposition(X):
        """Computes eigen decomposition of covariance matrix for data having
        the shape (features, examples), if features >> examples, uses computational trick
        to compute the eigen decomposition.

        Returns:
            ndarray of shape: (features, features)
        """
        num_features, N = X.shape
        mean = np.mean(X, axis=1)
        X = X - mean[:, None]

        if num_features > N:
            # use computational trick...
            u, s, vh = np.linalg.svd(X.T @ X)
            Q = X@vh.T
            Q = FaceClassifier.normalize(Q)
```

```
        else:
            u, s, vh = np.linalg.svd(X @ X.T)
            Q = FaceClassifier.normalize(vh.T)
        return s, Q


    @staticmethod
    def normalize(X):
        """Given the data matrix, normalizes each example separately.
        N is the total number of examples and m is the dimensions of each
            example.

        Args:
            X: ndarray = shape: (m, N)
        Returns:
            normalized X: (m, N)
        """
        X /= np.linalg.norm(X, axis=0)[None,:]
        return X




class DataBuilder(Dataset):
    def __init__(self, path):
        self.path = path
        self.image_list = [f for f in os.listdir(path) if f.endswith('.png')
            ]
        self.label_list = [int(f.split('_')[0]) for f in self.image_list]
        self.len = len(self.image_list)
        self.aug = transforms.Compose([
            transforms.Resize((64, 64)),
            transforms.ToTensor(),
        ])

    def __getitem__(self, index):
        fn = os.path.join(self.path, self.image_list[index])
        x = Image.open(fn).convert('RGB')
        x = self.aug(x)
        return {'x': x, 'y': self.label_list[index]}

    def __len__(self):
        return self.len


class Autoencoder(nn.Module):

    def __init__(self, encoded_space_dim):
        super().__init__()
        self.encoded_space_dim = encoded_space_dim
        ### Convolutional section
        self.encoder_cnn = nn.Sequential(
            nn.Conv2d(3, 8, 3, stride=2, padding=1),
            nn.LeakyReLU(True),
            nn.Conv2d(8, 16, 3, stride=2, padding=1),
            nn.LeakyReLU(True),
            nn.Conv2d(16, 32, 3, stride=2, padding=1),
            nn.LeakyReLU(True),
            nn.Conv2d(32, 64, 3, stride=2, padding=1),
```

```
            nn.LeakyReLU(True)
        )
        ### Flatten layer
        self.flatten = nn.Flatten(start_dim=1)
        ### Linear section
        self.encoder_lin = nn.Sequential(
            nn.Linear(4 * 4 * 64, 128),
            nn.LeakyReLU(True),
            nn.Linear(128, encoded_space_dim * 2)
        )
        self.decoder_lin = nn.Sequential(
            nn.Linear(encoded_space_dim, 128),
            nn.LeakyReLU(True),
            nn.Linear(128, 4 * 4 * 64),
            nn.LeakyReLU(True)
        )
        self.unflatten = nn.Unflatten(dim=1,
                                      unflattened_size=(64, 4, 4))
        self.decoder_conv = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 3, stride=2,
                               padding=1, output_padding=1),
            nn.BatchNorm2d(32),
            nn.LeakyReLU(True),
            nn.ConvTranspose2d(32, 16, 3, stride=2,
                               padding=1, output_padding=1),
            nn.BatchNorm2d(16),
            nn.LeakyReLU(True),
            nn.ConvTranspose2d(16, 8, 3, stride=2,
                               padding=1, output_padding=1),
            nn.BatchNorm2d(8),
            nn.LeakyReLU(True),
            nn.ConvTranspose2d(8, 3, 3, stride=2,
                               padding=1, output_padding=1)
        )

    def encode(self, x):
        x = self.encoder_cnn(x)
        x = self.flatten(x)
        x = self.encoder_lin(x)
        mu, logvar = x[:, :self.encoded_space_dim], x[:, self.
            encoded_space_dim:]
        return mu, logvar

    def decode(self, z):
        x = self.decoder_lin(z)
        x = self.unflatten(x)
        x = self.decoder_conv(x)
        x = torch.sigmoid(x)
        return x

    @staticmethod
    def reparameterize(mu, logvar):
        std = logvar.mul(0.5).exp_()
        eps = Variable(std.data.new(std.size()).normal_())
        return eps.mul(std).add_(mu)


class VaeLoss(nn.Module):
    def __init__(self):
```

```
        super(VaeLoss, self).__init__()
        self.mse_loss = nn.MSELoss(reduction="sum")

    def forward(self, xhat, x, mu, logvar):
        loss_MSE = self.mse_loss(xhat, x)
        loss_KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
        return loss_MSE + loss_KLD




def get_vae_features(p, face_pix_dir, weights_dir):

    # training = False
    TRAIN_DATA_PATH = os.path.join(face_pix_dir, 'train')
    EVAL_DATA_PATH = os.path.join(face_pix_dir, 'test')
    LOAD_PATH = os.path.join(weights_dir, f'model_{p}.pt')
    # OUT_PATH = output_dir
    ###################################

    model = Autoencoder(p)

    trainloader = DataLoader(
        dataset=DataBuilder(TRAIN_DATA_PATH),
        batch_size=1,
    )
    model.load_state_dict(torch.load(LOAD_PATH))
    model.eval()

    X_train, y_train = [], []
    for batch_idx, data in enumerate(trainloader):
        mu, logvar = model.encode(data['x'])
        z = mu.detach().cpu().numpy().flatten()
        X_train.append(z)
        y_train.append(data['y'].item())
    X_train = np.stack(X_train)
    y_train = np.array(y_train)

    testloader = DataLoader(
        dataset=DataBuilder(EVAL_DATA_PATH),
        batch_size=1,
    )
    X_test, y_test = [], []
    for batch_idx, data in enumerate(testloader):
        mu, logvar = model.encode(data['x'])
        z = mu.detach().cpu().numpy().flatten()
        X_test.append(z)
        y_test.append(data['y'].item())
    X_test = np.stack(X_test)
    y_test = np.array(y_test)

    return X_train.T, y_train, X_test.T, y_test




class ObjectDetector:
    def __init__(self, data_dir) -> None:
        self.data_dir = data_dir
```

```
375
376         self.train_images, self.train_y = self.read_images(True)
377
378         self.test_images, self.test_y = self.read_images(False)
379
380
381     def test_adaBoost(self, stage_classifiers_list, stage_alphas_list,
        threshold = 0.5):
382
383         test_labels = self.test_y
384         test_features = self.get_features(False)
385         pos_test_feats = test_features[np.where(test_labels==1)].T
386         neg_test_feats = test_features[np.where(test_labels==0)].T
387
388         pos_labels = test_labels[np.where(test_labels==1)]
389         neg_labels = test_labels[np.where(test_labels==0)]
390         num_pos = pos_labels.shape[0]
391         num_neg = neg_labels.shape[0]
392
393         stage_cl = stage_classifiers_list
394         stage_al = stage_alphas_list
395
396         test_feats = np.concatenate([pos_test_feats, neg_test_feats], axis
            =1)
397
398         pred_arr = []
399
400         # iterate through every stage's classifiers
401         for classifiers, alphas in zip(stage_cl, stage_al):
402             weak_preds = []
403             # print(len(cls), len(als))
404             # iterate through each weak classifier
405             for cl in classifiers:
406                 # unpack classifier params
407                 i, thresh, pol, _ = cl
408                 # go to feature index in test samples
409                 current_feat = test_feats[int(i)]
410                 # classify
411                 test_pred = current_feat >= thresh if pol == 1 else
                        current_feat <= thresh
412                 weak_preds.append(np.array(test_pred).astype(np.uint8))
413             weak_preds = np.transpose(np.array(weak_preds))
414
415             strong_out = np.dot(weak_preds, alphas)
416             thr = np.sum(alphas) * threshold
417             strong_pred = np.zeros_like(strong_out)
418             strong_pred[strong_out >= thr] = 1
419             pred_arr.append(strong_pred)
420
421         pred_arr = np.array(pred_arr)
422
423         FP_list = []
424         FN_list = []
425
426         # evaluate predicted labels
427         num_stages = pred_arr.shape[0]
428         stage_preds = []
429         stage_preds.append(pred_arr[0])
430         for i in range(1, num_stages):
```

```
            stage_preds.append(np.logical_and(stage_preds[-1], pred_arr[i]))
            # this loop removes correctly classified negative examples from
                the pred label set
            # for j in range(i+1):
                # ps_= np.logical_and(ps_,pred_arr[j])
            # ref_pred = ps_
            FP = np.sum(stage_preds[-1][np.where(test_labels==0)])/
                stage_preds[-1][np.where(test_labels==0)].size
            TP = np.sum(stage_preds[-1][np.where(test_labels==1)])/
                stage_preds[-1][np.where(test_labels==1)].size
            FP_list.append(FP)
            FN_list.append(1 - TP)

        return FP_list, FN_list




    def cascaded_adaBoost(self, max_stages=20):

        train_features = self.get_features(True)
        train_labels = self.train_y

        pos_labels = train_labels[np.where(train_labels==1)]
        neg_labels = train_labels[np.where(train_labels==0)]

        pos_features = train_features[np.where(train_labels==1)].T
        neg_features = train_features[np.where(train_labels==0)].T

        stage_cl_list = []
        stage_alpha_list = []

        FP_list = []
        for stage in range(max_stages):
            neg_features, neg_labels, classifiers_list, alphas_list, fpr =
                self.get_strong_classifier(
                pos_features, neg_features, pos_labels, neg_labels
                )

            stage_cl_list.append(classifiers_list)
            stage_alpha_list.append(alphas_list)
            # append fpr rates while training
            FP_list.append(fpr)
            # if there are no more negative samples, stop training
            if neg_features.shape[1] == 0:
                print(f'Training Stopped at Stage: { stage + 1}')
                break
        return stage_cl_list, stage_alpha_list, FP_list


    def read_images(self, train=True):
        if train:
            split = 'train'
        else:
            split = 'test'

        images = []
        labels = []

```

```
        for cat in ['positive', 'negative']:
            cat_images = []
            filenames = os.listdir(os.path.join(self.data_dir, split, cat))
            for filename in filenames:
                img = read_image_from_path(os.path.join(self.data_dir, split
                    , cat, filename))
                img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
                cat_images.append(img)
            if cat == 'positive':
                labels.append(np.ones(len(cat_images)))
            else:
                labels.append(np.zeros(len(cat_images)))
            images.extend(cat_images)
        images = np.array(images).astype(np.float32)
        labels = np.concatenate(labels).astype(np.float32)
        return images, labels

    def get_features(self, train=True, force_redo=False):
        if train:
            saved_filename = f"haar_features_train.npz"
        else:
            saved_filename = f"haar_features_test.npz"

        filepath = os.path.join(self.data_dir, saved_filename)
        if not os.path.exists(filepath) or force_redo:
            print(f"Extracting haar features for train={train}")
            features = self.haar_features_all_images(train)
            np.savez_compressed(filepath, features=features)
        else:
            print(f"Reading features from memory for train={train}")
            features = np.load(filepath)['features']
        return features

    def haar_features_all_images(self, train=True):

        if train:
            images = self.train_images
        else:
            images = self.test_images
        haar_features = []
        for img in images:
            haar_features.append(self.haar_features_for_img(img))
        return np.stack(haar_features)

    @staticmethod
    def haar_features_for_img(image):

        features = []
        height, width = image.shape
        horz_windows = np.arange(2, width, 2)
        for hw in horz_windows:
            for i in range(height):
                for j in range(hw//2, width-hw//2):
                    neg_sum = np.sum(image[i, j-hw//2:j])
                    pos_sum = np.sum(image[i, j:j+hw//2])
                    features.append(pos_sum - neg_sum)
        vert_windows = np.arange(2, height, 2)
        for vw in vert_windows:
            for i in range(vw//2, height-vw//2):
```

```
                for j in range(width):
                    neg_sum = np.sum(image[i-vw//2:i, j])
                    pos_sum = np.sum(image[i:i+vw//2, j])
                    features.append(pos_sum - neg_sum)
        return np.array(features)

    @staticmethod
    def get_weak_classifier(weight, features, labels):
        """Returns the best weak classifier using the haar features and
        weights.
        """
        # making sure the weights are normalized..
        weight /= np.sum(weight)
        least_err = np.inf

        Tp = np.sum(weight[np.where(labels == 1)])
        Tn = np.sum(weight[np.where(labels == 0)])

        idx = np.argsort(features, axis=1)
        # sort features, weights and labels according to idx
        sorted_feats = np.take_along_axis(features, idx, axis=1)
        sorted_weight = np.take_along_axis(weight[None,:], idx, axis=1)
        sorted_labels = np.take_along_axis(labels[None,:], idx, axis=1)

        Sp = np.cumsum(sorted_weight * sorted_labels, axis=1)
        Sn = np.cumsum(sorted_weight, axis=1) - Sp

        error_type1 = Sp + (Tn - Sn)
        error_type2 = Sn + (Tp - Sp)

        best_weak_classifier = None
        f_pred = None
        for ii, feat in enumerate(features):

            min_error = np.minimum(error_type1[ii], error_type2[ii])
            min_idx = np.argmin(min_error)
            if error_type1[ii][min_idx] <= error_type2[ii][min_idx]:
                p = 1
                pred = (feat >= sorted_feats[ii][min_idx])
            else:
                p = -1
                pred = (feat < sorted_feats[ii][min_idx])
            err = min_error[min_idx]
            if err < least_err:
                best_weak_classifier = [ii, sorted_feats[ii][min_idx], p,
                    err]
                f_pred = pred.astype(np.uint8)
                least_err = err
        return best_weak_classifier, f_pred

    @staticmethod
    def get_strong_classifier(pos_features, neg_features, pos_labels,
        neg_labels, req_TP=1, req_FP=0.45, max_num_classifiers=50):
        classifiers_list = []
        alphas_list = []
        preds_list = []
        # tpr, fpr, tnr, fnr, rep_fpr = 0, 0, 0, 0, 0
        num_pos = pos_features.shape[1]
        num_neg = neg_features.shape[1]
```

```
        pos_weights = np.ones(num_pos)/(2*num_pos)
        neg_weights = np.ones(num_neg)/(2*num_neg)
        weights = np.concatenate([pos_weights, neg_weights])
        feats = np.concatenate((pos_features, neg_features), axis=1)
        labels = np.concatenate([pos_labels, neg_labels])

        for n in range(max_num_classifiers):
            best_weak_cl, f_pred = ObjectDetector.get_weak_classifier(
                weights, feats, labels)
            classifiers_list.append(best_weak_cl)

            error = best_weak_cl[-1]
            alpha = 0.5*np.log((1-error)/error)
            alphas_list.append(alpha)
            # Eq. 5 in AdaBoost slides gives weights update
            # in terms of y*h and required y*h=1 for matching prediction
            # and y*h=-1 for mismatched predictions.
            label_times_pred = np.ones_like(labels)
            label_times_pred[labels!=f_pred] = -1
            weights = weights * np.exp(-alpha*label_times_pred)

            preds_list.append(f_pred)
            feat_arr = np.transpose(np.array(preds_list))
            alpha_arr = np.transpose(np.array([alphas_list]))
            classifier_out = np.dot(feat_arr, alpha_arr)
            threshold = np.min(classifier_out[:num_pos])
            classifier_pred = np.zeros_like(classifier_out)

            classifier_pred[classifier_out >= threshold] = 1

            # calculate true positive and false positive rates
            tpr = np.sum(classifier_pred[np.where(labels==1)]) /
                classifier_pred[np.where(labels==1)].size
            fpr = np.sum(classifier_pred[np.where(labels==0)]) /
                classifier_pred[np.where(labels==0)].size
            FP_entire_data = np.sum(classifier_pred[np.where(labels==0)]) /
                1758

            if tpr >= req_TP and fpr <= req_FP:
                print(f'Num Classifier in stage is { n + 1}')
                break

        neg_preds = classifier_pred[num_pos:]
        FP_ids = np.where(neg_preds==1)[0] ###
        FP_example_features = neg_features[:, FP_ids]
        FP_example_labels = np.zeros(len(FP_ids))
        print(f'False Postives in stage = { len(FP_ids)}')

        return FP_example_features, FP_example_labels, np.array(
            classifiers_list), np.array(alphas_list), FP_entire_data




if __name__ == '__main__':

    # Task - 1 & 2
```

```
    weights_dir = r'C:\Users\ahmedb\projects\computer-vision\Auxilliary\hw10-vae-weights'
    face_pix_dir = r'C:\Users\ahmedb\projects\computer-vision\Auxilliary\FaceRecognition'
    output_dir = r'C:\Users\ahmedb\projects\computer-vision\images\hw10'

    logger.info(f"Task-1: Face recognition using PCA and LDA...")
    classifier = FaceClassifier(face_pix_dir, output_dir)

    p_values = np.arange(1, 21)
    pca_acc_list = []
    lda_acc_list = []
    for k in p_values:
        W_k = classifier.get_PCA_space(k=k)
        predicted_y = classifier.NN_predictor(W_k)
        acc = classifier.compute_accuracy(predicted_y)
        pca_acc_list.append(acc)

        W_k = classifier.get_LDA_space(k=k)
        predicted_y = classifier.NN_predictor(W_k)
        acc = classifier.compute_accuracy(predicted_y)
        lda_acc_list.append(acc)

        if k in [3,8,16]:
            classifier.visualize_embeddings(k=k)

    logger.info(f"Task-2: Face recognition using Autoencoder...")
    vae_acc_dict = {}
    for p in [3,8,16]:
        X_train, y_train, X_test, y_test = get_vae_features(p, face_pix_dir, weights_dir)
        predicted_y = FaceClassifier.NN(X_train, y_train, X_test)
        acc = classifier.compute_accuracy(predicted_y)
        vae_acc_dict[p] = acc

        FaceClassifier.plot_umap_embeddings(X_train.T, y_train)
        plt.title(f'UMAP projected training data using VAE-{p}')
        plt.savefig(os.path.join(output_dir, f'vae_umap_k_{p}_train.png'))

        FaceClassifier.plot_umap_embeddings(X_test.T, predicted_y)
        plt.title(f'UMAP projected test data using VAE-{p}')
        plt.savefig(os.path.join(output_dir, f'vae_umap_k_{p}_test.png'))

    plt.subplots()
    plt.plot(p_values, pca_acc_list, label='PCA', marker='o')
    plt.plot(p_values, lda_acc_list, label='LDA', marker='x')
    plt.plot(vae_acc_dict.keys(), vae_acc_dict.values(), label='VAE', marker='*')
    plt.ylabel("accuracy (%)")

    plt.xticks(p_values,p_values)
    plt.xlabel(f"sub-space dimensions (p)")
    plt.legend(loc='best')
    plt.title(f"Face detection accuracy using different methods")
    plt.savefig(os.path.join(output_dir, 'accuracy.png'))


    # Task - 3
    logger.info(f"Task-3: Car detection using cascaded adaBoost...")
```

```
    car_data_dir = r'C:\Users\ahmedb\projects\computer-vision\Auxilliary\
        CarDetection'
    detector = ObjectDetector(car_data_dir)

    stage_cl_list, stage_alpha_list, training_FP_list = detector.
        cascaded_adaBoost()



    FP_list, FN_list = detector.test_adaBoost(stage_cl_list,
        stage_alpha_list)

    stages = np.arange(1, len(training_FP_list)+1)
    training_FP_list = np.array(training_FP_list)*100
    plt.subplots()
    plt.plot(stages, training_FP_list)
    plt.xticks(stages, stages)
    plt.xlabel(f"stages of cascade")
    plt.ylabel(f"False postive rate (%)")
    plt.title(f"Training false positive rate")
    plt.savefig(os.path.join(output_dir, 'training_fpr.png'))

    stages = np.arange(1, len(FP_list)+1)
    FP_list = np.array(FP_list)*100
    FN_list = np.array(FN_list)*100
    plt.subplots()
    plt.plot(stages, FP_list, label='FP')
    plt.plot(stages, FN_list, label='FN')
    plt.legend(loc='best')
    plt.xticks(stages, stages)
    plt.xlabel(f"stages of cascade")
    plt.ylabel(f"percentage (%)")
    plt.title(f"Performance on test set")
    plt.savefig(os.path.join(output_dir, 'testing_fpr_fnr.png'))
```

Listing 1: Example Python code